



STRENGTH THROUGH SYSTEMS

ONE PROMPT. ONE APP. ONE LIVE LINK.

Build the tool your facility actually needs, put it on the internet, and run it off a link — without writing a line of code yourself.

- 01 **The Prompt** — turn a problem into a working index.html
- 02 **GitHub** — give the file a permanent home
- 03 **Netlify** — put it online in under five minutes
- 04 **Supabase** — the bonus upgrade past local storage

START HERE

READ THIS FIRST

Most facilities don't have a software problem. They have a **system problem** that software would fix in an afternoon — if anyone knew how to build it. Attendance on a clipboard. Maxes in a notebook. Programs in someone's head. A spreadsheet that only one person can open without breaking it.

You can build the tool yourself. Not "learn to code for two years" build it. **Describe it clearly, deploy it, use it Monday.**

WHAT YOU'RE GOING TO END UP WITH

One file — `index.html` — that contains your entire app: the screens, the logic, the styling, the data. It opens in any browser, on any tablet at any rack, and it lives at a real web address you can share with your staff.

THE CORE PRINCIPLE

Clarity in, clarity out. AI does not fix a fuzzy idea — it multiplies it. The quality of your app is decided **before** you hit enter, by how precisely you can describe the problem, the user, and the data. That's a coaching skill, not a coding skill. You already have it.

WHAT YOU NEED

- A Claude account (free tier works; paid handles bigger files better)
- An email address you can check
- About 45 minutes for the first build, 5 minutes for every deploy after
- Zero dollars. Every tool in this guide has a free tier that covers a single facility.

THE FOUR PARTS

PART	WHAT IT DOES	TIME
01 — The Prompt	Turns your problem into a working single-file app	20 min
02 — GitHub	Stores the file, keeps every version, never loses it	10 min
03 — Netlify	Turns the file into a live link your staff can open	5 min
04 — Supabase	Bonus: moves data off one device so a whole staff shares it	30 min

DO THIS IN ORDER

Get Parts 1–3 done and **use the app for two weeks on local storage first**. You will change your mind about half the features. Changing your mind is cheap before there's a database and free after — but only if you didn't build the database around the wrong idea.

THE CLAUDE PROMPT

One prompt. Fill in the blanks. Paste it into Claude.

This prompt is built to force three things that make or break a single-file app: **one file with no build step**, a **named data model**, and a **hard feature boundary**. Skip any of the three and you get a demo. Include all three and you get a tool.

Everything in **[BRACKETS]** is yours to replace. Everything else, leave exactly as written.

ROLE

You are a senior front-end engineer who ships complete, working software. No placeholders. No "TODO". No "rest of the code here". You write the entire file, every time.

WHAT TO BUILD

A single file called index.html – a working [APP NAME] for [WHO USES IT].

THE PROBLEM

[2-3 sentences in plain language. What is broken today, who it slows down, and what has to get faster. Be specific. "Coaches write attendance on a clipboard, then nobody enters it, so I can't tell a parent how many sessions their kid actually showed up for."]

WHO USES IT AND WHERE

[Example: one coach, on a tablet, standing on the floor mid-session, one hand free, in a hurry. Or: the owner, at a desk, once a week.] Design every screen around that person and that moment.

MODES – these are the only top-level tabs

1. [MODE NAME] – [one line: what a user does here]
2. [MODE NAME] – [one line]
3. [MODE NAME] – [one line]

Build only these. Do not add features I did not ask for. If you think something is missing, list it at the end instead of building it.

DATA MODEL

All data lives in browser localStorage. Use one namespace prefix: [app:]

- [app:athletes] -> array of { id, name, group, sport }
- [app:sessions] -> array of { id, date, athleteId, status }
- [describe every list of records the app stores, and its fields]

Rules for data:

- Every record gets a stable unique id (crypto.randomUUID()).
- Every read and write goes through two functions only: load(key) and save(key, value). Nothing else touches localStorage directly.
- Dates stored as YYYY-MM-DD strings, never Date objects.
- If a key is missing or corrupted, fall back to an empty list and keep running. The app must never show a blank white screen.

THE STANDARDS

Same prompt, second half. This is the part that separates a tool from a toy.

NON-NEGOTIABLE REQUIREMENTS

- ONE file. All HTML, CSS and JavaScript inline in index.html.
- Vanilla JavaScript only. No React, no npm, no build step, no bundler.
- No external requests except fonts. It must work with no internet after first load, and it must run by double-clicking the file.
- Tablet and phone first: minimum 44px tap targets, no hover-only controls, no tiny text, thumb-reachable primary buttons.
- Data survives a refresh and a browser close.
- Every screen that holds data gets an "Export CSV" button.
- One "Backup / Restore" control that exports all app data as a single JSON file and can import it back.
- Nothing destructive without a confirm step.
- Every empty screen shows a one-line empty state that tells the user what to do first – never a blank panel.
- Zero console errors on load or on any click.

DESIGN

- Background [#000000], accent [#13BE17], text [#FFFFFF].
- Headers uppercase, bold, tight letter spacing. Body clean and plain.
- High contrast, minimal, no gradients, no rounded-everything, no icons that need explaining.
- Define every color as a CSS variable in :root so I can rebrand the whole app by editing six lines.

BUILD RULES

- Write the file top to bottom, complete, in one code block.
- Mark sections with comment headers I can search for:
`/* ===== [MODE NAME] ===== */`
- Small named functions. A function name should say what it does.
- Where something was ambiguous, pick the simplest option that works and list those choices at the end – three bullets, maximum.

OUTPUT

The complete index.html in one code block, then a five-line "how to use this" note. Nothing else.

WHY THESE ARE NON-NEGOTIABLE

Every line in that block is there because leaving it out creates work later — a file you can't open offline, data you can't get out, a delete button with no confirm, a screen a coach can't hit with a thumb.

Standards before complexity.

RUNNING THE BUILD

1 FILL IN THE BRACKETS BEFORE YOU OPEN CLAUDE

Write your answers in a notes app first. If you can't describe the modes and the data model on paper, the prompt will not save you. Ten minutes here saves an hour of re-prompting.

2 PASTE IT AND LET IT RUN

Claude will write the full file. On a big app it may pause partway through. Reply with exactly: *"Continue from where you stopped. Do not restate anything you already wrote."*

3 SAVE IT AS INDEX.HTML

Lowercase. Exactly that name. It matters later — that filename is what tells a web host which page to open first.

4 DOUBLE-CLICK IT AND ACTUALLY USE IT

It opens in your browser. Add three real athletes. Log a real session. Refresh the page and confirm the data is still there. Anything that annoys you in the first five minutes will annoy your staff every day — write it down.

THE TWO PROMPTS YOU'LL USE FOREVER

Fixing something:

```
Here is my current index.html: [paste the whole file]

Problem: [what you clicked, what you expected, what happened instead].

Fix only that. Change nothing else. Return the complete updated file.
```

Adding something:

```
Here is my current index.html: [paste the whole file]

Add: [one feature, described the same way as a mode above — what the
user does, what data it reads, what data it writes].

Keep everything else exactly as it is, including the data model and the
existing localStorage keys. Return the complete updated file.
```

COMMON MISTAKE

Asking for five changes in one message. You'll get four of them, one silent regression, and no idea which change caused it. **One change per prompt. Test. Save. Then the next one.** Same principle as changing one training variable at a time.

WHEN THE FILE GETS BIG

Past roughly 2,000 lines, pasting the whole file gets slow and lossy. Switch to surgical edits: *"Return ONLY the updated version of the function `renderRoster()`, plus a one-line note on exactly which lines to replace."*

Where the file lives, and how you stop losing versions.

GitHub is a filing cabinet with a memory. Every time you save a change, it keeps the old copy forever. Break something on a Tuesday and you can be back to Monday's working version in thirty seconds. It's also the pipe that feeds your live site in Part 3 — save here, and the site updates itself.

1 CREATE THE ACCOUNT

Go to github.com → Sign up. Use an email you'll keep. Verify it. Free plan, no card.

2 CREATE A REPOSITORY

Top right + → **New repository**.

- **Name:** lowercase with dashes — `facility-attendance`. No spaces, no capitals.
- **Public or Private:** Private is fine — Netlify's free tier connects to private repos. Choose Private if the app will ever hold athlete names.
- Check **Add a README file**. Leave everything else alone.
- Click **Create repository**.

3 UPLOAD YOUR FILE

Add file → **Upload files** → drag `index.html` in. In the "Commit changes" box, type what you did — *"First working version"* — then **Commit changes**.

4 EVERY UPDATE AFTER THIS

Click `index.html` in the repo → the pencil icon → select all, paste your new version over it → **Commit changes** with a note. That note is your changelog. Write real ones: *"Added CSV export to roster"* beats *"update"* when you're hunting for the version that still worked.

ROLLING BACK A BAD CHANGE

Open `index.html` in the repo → **History** → click the commit from before things broke → **View file** → copy it → paste it back over the current version and commit. You just undid a bad day in about a minute.

COMMON MISTAKE

Uploading the file inside a folder, or naming it `Index.html`, `index.HTML`, or `app.html`. It must sit at the top level of the repo and be named `index.html`, lowercase. Get this wrong and Part 3 gives you a blank page with no error to explain it.

Turning the file into a link you can text to your staff.

Netlify watches your GitHub repo. When the file changes, it publishes the new version automatically — usually in under thirty seconds. You set this up once and never think about it again.

1 SIGN UP WITH GITHUB

`netlify.com` → Sign up → **Sign up with GitHub**. Using GitHub to sign in means the two accounts are already connected. Free plan, no card.

2 IMPORT THE REPO

Add new site → **Import an existing project** → **GitHub** → authorize → pick your repository. If you don't see it, click *Configure Netlify on GitHub* and grant access to that repo.

3 LEAVE THE BUILD SETTINGS EMPTY

This is the step people overthink. You have no build step — it's one plain HTML file.

- **Build command:** leave blank
- **Publish directory:** leave blank (or a single dot: `.`)

Click **Deploy**. Wait. You now have a live app.

4 NAME IT SOMETHING YOU CAN SAY OUT LOUD

Netlify gives you a random name like `zesty-tiger-4a1b9c`. Change it: **Site configuration** → **Change site name** → `your-facility-attendance`. Your link is now `your-facility-attendance.netlify.app`.

5 PUT IT ON THE TABLETS

Open the link on each tablet → browser menu → **Add to Home Screen**. It gets an icon and opens full screen with no address bar. To your staff it's just an app.

HOW UPDATES WORK FROM HERE

Edit `index.html` in GitHub → commit → Netlify redeploys on its own. No re-uploading, no re-installing, no "make sure everyone has the new version." Everyone opens the same link and always has the latest build.

KNOW THIS BEFORE YOU TRUST IT

Until you finish Part 4, **the data lives in the browser, not on the internet**. The link is shared; the data is not. Coach A's tablet and Coach B's phone are two separate sets of records. Clearing browser data wipes it. Live with that on purpose for two weeks — then upgrade.

SUPABASE

04

One shared source of truth instead of six tablets that disagree.

Local storage is the right call for one coach on one device. It stops being the right call the second a second coach needs the same numbers. Supabase is a free hosted database that fixes that without you managing a server.

UPGRADE WHEN ANY OF THESE ARE TRUE

- More than one coach enters data
- You want the same data on the floor tablet and the office laptop
- You're generating reports that leave the building — parents, ADs, owners
- Losing the data would cost you a real conversation you can't have twice

THE PATTERN THAT KEEPS THIS SIMPLE

Do **not** rebuild the app around a database. Keep local storage as the app's working memory and **mirror it up to one table**. The app stays fast and still works when the wifi drops in the corner of the weight room. The cloud becomes a sync layer, not a dependency.

1 CREATE THE PROJECT

supabase.com → Start your project → sign in with GitHub → **New project**. Name it, pick the region closest to your facility, and **save the database password somewhere permanent** — it is not shown again.

2 GRAB YOUR TWO KEYS

Project Settings → **API**. Copy the **Project URL** and the **anon public** key. Those two strings are all your app needs.

3 CREATE THE TABLE

Open **SQL Editor** → New query → paste the block on the next page → **Run**.

4 TURN ON EMAIL LOGIN

Authentication → **Providers** → Email → enabled. Then **Users** → **Add user** and create an account for each coach by hand. For a staff of four, manual beats a sign-up flow you have to police.

THE ONE SECURITY RULE

The anon key is **meant** to be visible in your file — that's normal and not a leak. What protects your data is **Row Level Security**, which is what the SQL on the next page turns on. Never paste the `service_role` key into `index.html`. That one is a master key, and anyone who views your page source would have it.

THE TABLE & THE RULES

Deliberately one table. Your app already organizes its data under named keys — this stores each key as a row, scoped to your facility and locked to your staff.

```
-- 1. Facilities and who belongs to them
create table facilities (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  created_at timestamptz default now()
);

create table facility_members (
  facility_id uuid references facilities(id) on delete cascade,
  user_id uuid references auth.users(id) on delete cascade,
  role text not null default 'coach',
  primary key (facility_id, user_id)
);

-- 2. One row per app data key
create table app_documents (
  id uuid primary key default gen_random_uuid(),
  facility_id uuid not null references facilities(id) on delete cascade,
  doc_key text not null,
  data jsonb not null,
  updated_by uuid,
  updated_at timestamptz default now(),
  unique (facility_id, doc_key)
);

-- 3. Lock it down – nobody sees another facility's data
alter table facilities enable row level security;
alter table facility_members enable row level security;
alter table app_documents enable row level security;

create policy "members read their facility" on facilities
  for select using (
    id in (select facility_id from facility_members
          where user_id = auth.uid())
  );

create policy "members read their membership" on facility_members
  for select using (user_id = auth.uid());

create policy "members read docs" on app_documents
  for select using (
    facility_id in (select facility_id from facility_members
                   where user_id = auth.uid())
  );

create policy "members write docs" on app_documents
  for all using (
    facility_id in (select facility_id from facility_members
                   where user_id = auth.uid())
  );
```

After it runs: open the **Table Editor**, add one row to **facilities** with your facility name, then a row in **facility_members** for each coach — that facility's **id** paired with the user's **id** from the Authentication tab.

PRACTICAL APPLICATION

Your whole staff now sees the same roster, maxes and attendance — and physically cannot see another facility's. That last part is why this structure scales from one weight room to twenty.

WIRING IT INTO THE APP

Here is my current index.html: [paste the whole file]

Add a cloud sync layer to this app using Supabase. Do not rewrite the app. Do not change how any existing screen works.

KEEP LOCAL STORAGE AS THE SOURCE OF TRUTH. Every existing feature must keep reading and writing localStorage exactly as it does now, and the app must keep working with no internet.

Add on top of that:

1. Load the Supabase client from the CDN. Put my credentials in a single config object at the very top of the script so I can find and change them in one place:
`SUPABASE_URL = "[paste your project URL]"`
`SUPABASE_ANON_KEY = "[paste your anon public key]"`
2. A login screen shown before the app. Email and password against Supabase Auth. On success, look up the user's facility_id from the facility_members table and hold it in memory. Add a Sign Out button in the header showing who is logged in.
3. On login, PULL: read every row from app_documents for that facility_id and write each row's data into localStorage under its doc_key. Then render the app.
4. On any change, PUSH: wrap localStorage.setItem so that whenever a synced key changes, it upserts into app_documents as { facility_id, doc_key, data, updated_by, updated_at }. Debounce about 700ms so a burst of edits becomes one write.
5. Subscribe to realtime changes on app_documents for that facility_id. When another device writes, update localStorage and re-render — but ignore changes this device made.
6. If Supabase is unreachable or the credentials are blank, fall back to local-only mode silently. The app must never break because the internet is down.
7. Show a status indicator in the header: SYNCED / OFFLINE.

Only these keys sync: [list your app: keys here].

Return the complete updated index.html in one code block.

TEST IT LIKE A COACH, NOT A DEVELOPER

Two devices, two coaches. Add an athlete on one — it should appear on the other in seconds. Then kill the wifi on one, add a session, turn it back on, confirm it lands.

COMMON MISTAKE

Wiring in the cloud before the app is settled. **Two weeks of real floor use first** — then sync. The pattern has to be stable before you load it.

WHEN IT BREAKS

Nine problems account for almost everything that goes wrong.

WHAT YOU SEE

WHAT IT ACTUALLY IS

Live site is blank

The file isn't named `index.html` (lowercase), or it's sitting inside a folder instead of at the top of the repo.

Netlify says the build failed

Something got typed into the build command. Clear it. Site configuration → Build & deploy → empty build command, publish directory `.`

Changes aren't showing up

Check the Deploys tab for a green "Published." If it's green, hard refresh: Ctrl+Shift+R, or Cmd+Shift+R on Mac.

Data vanished

Local storage is per browser, per device. Different browser, private window, or cleared history means a different, empty box. This is Part 4's whole reason to exist.

Claude stopped mid-file

Reply: "Continue from where you stopped. Do not restate anything you already wrote." Then join the pieces in a text editor.

A fix broke two other things

You changed several things at once. Roll back in GitHub history, then redo it one change at a time.

Supabase returns a 401 or empty results

Almost always RLS: the logged-in user has no row in `facility_members`, or you're querying with the wrong `facility_id`.

Login works, no data appears

The pull ran before the facility lookup finished. Tell Claude: "Wait for the `facility_id` lookup to resolve before pulling documents."

App feels slow on the tablet

Something re-renders the whole screen on every keystroke. Tell Claude: "Only re-render the panel that changed, not the whole view."

THE HABIT THAT PREVENTS MOST OF THIS

Commit to GitHub **every time the app works**, not every time you finish. A working version you can return to is worth more than a clever version you can't reproduce.

THE CHECKLIST

Print this. Work down it. Don't skip ahead.

PART 01 — THE PROMPT

- ☐ Problem written in plain language, three sentences or fewer
- ☐ User and moment named — who, where, holding what
- ☐ Modes listed, one line each, nothing extra
- ☐ Data model written out: keys and fields
- ☐ Prompt run, file saved as `index.html`
- ☐ Opened it, entered real data, refreshed, data survived

PART 02 — GITHUB

- ☐ Account created and email verified
- ☐ Repository created (Private if it holds athlete names)
- ☐ `index.html` uploaded at the top level, lowercase
- ☐ First commit written in real words

PART 03 — NETLIFY

- ☐ Signed up with GitHub
- ☐ Repo imported, build command left blank
- ☐ Site deployed and opening on a phone
- ☐ Site renamed to something sayable
- ☐ Added to the home screen on every floor tablet

PART 04 — SUPABASE (AFTER TWO WEEKS OF REAL USE)

- ☐ Project created, database password stored somewhere permanent
- ☐ SQL run, RLS on, policies created
- ☐ Coach accounts created, members rows added
- ☐ Sync prompt run, keys pasted in
- ☐ Two-device test passed, offline test passed

LAST THING

THE POINT OF ALL THIS

You didn't build software. You built a **system that holds its shape when you're not standing in the room**. That's the whole difference between a facility that runs on people remembering and a facility that runs on structure.

The clipboard works right up until the coach holding it takes a different job. The spreadsheet works right up until two people open it. A tool with one link, one data model, and one set of standards works on a Tuesday in January when you're out sick and a sub is running the floor.

WHAT TO BUILD FIRST

Not the big one. Build the smallest thing that removes the most repeated friction — usually attendance, or a single testing entry screen. Ship it in a week. Use it for a month. Let the second app be obvious because the first one made it obvious.

THE STANDARD

If it can't be coached, repeated, and handed to someone else, it isn't a system yet — it's a habit that only works when you're there. Same rule in the weight room. Same rule here.



SYSTEMS DRIVE SUCCESS

Turn chaos into clarity — on the floor and in the file.